Spark DataFrame



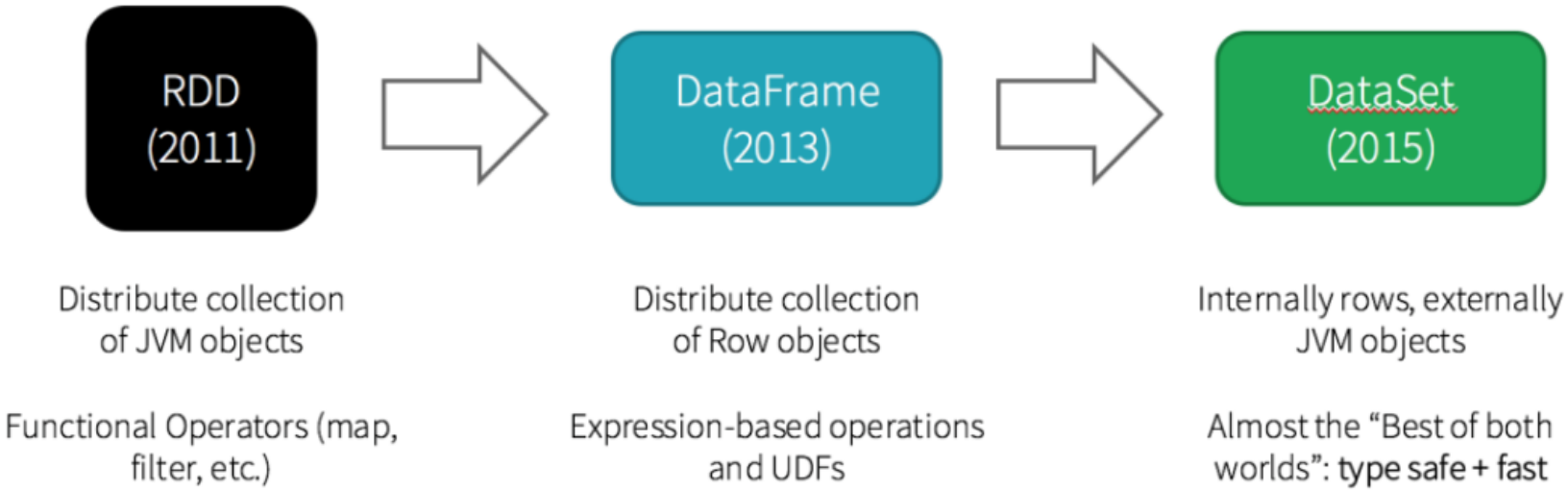
Spark DataFrame

A Dataset is a distributed collection of data.

❑ Spark DataFrame 介绍

➢ 官方原文：A DataFrame is a DataSet organized into named columns.

中文翻译：以列（列名，列类型，列值）的形式构成的分布式的数据集。



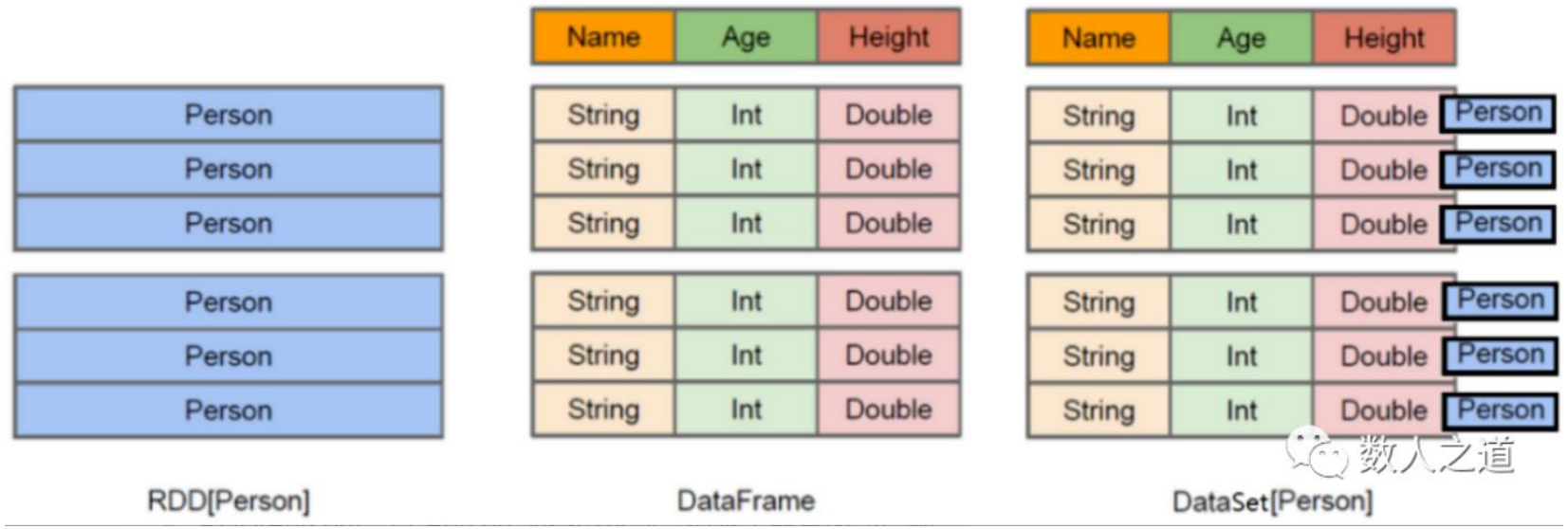
Source: https://sqlwithmanoj.com/2020/10/23/apache-spark-rdd-dataframes-transformations-narrow-wide-actions-lazy-evaluation-part-3/

Spark DataFrame

❑ Spark DataFrame 介绍

➢ RDD、DataFrame、DataSet 在结构上的区别

- RDD[Person]：以 Person 为类型参数，但不了解其内部结构。
- DataFrame：提供了详细的结构信息 Schema，包括列的名称和数据类型，类似于传统数据库的二维表。
- DataSet[Person]：不单有 Schema 信息，还有类型信息



Source: https://cloud.tencent.com/developer/article/1965056

Spark DataFrame

❑ Spark DataFrame 介绍

➢ Spark DataFrame 是Spark 2.0 之后使用Spark和python的主要工作方式。可以理解为功能强大的“表格”，具有行和列，用来处理大型数据集。

- 更简单的语法
- 能够直接使用SQL
- 操作自动分布在RDD上

Spark DataFrame

❑ Spark DataFrame – 代码部分

➢ Spark DataFrame Documentation Example

```
# 创建 DataFrame
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName('Basics').getOrCreate()

df = spark.read.json('file:///home/… …/people.json')
df.show()
```

如果报错，
使用绝对路径

```
+----+-------+
| age|   name|
+----+-------+
| NaN|Michael|
|30.0|   Andy|
|19.0| Justin|
+----+-------+
```

Spark DataFrame

❑ Spark DataFrame – 代码部分

➢ Spark DataFrame Documentation Example

```
df.printSchema()      # 查看数据结构
```

```
root
 |-- age: double (nullable = true)
 |-- name: string (nullable = true)
```

```
df.columns
```

```
['age', 'name']
```

```
df.describe()
```

```
DataFrame[summary: string, age: string, name: string]
```

```
df.describe().show()
```

```
+-------+----+-------+
|summary| age|   name|
+-------+----+-------+
|  count|   3|      3|
|   mean| NaN|   null|
| stddev| NaN|   null|
|    min|19.0|   Andy|
|    max| NaN|Michael|
+-------+----+-------+
```

Spark DataFrame

❑ Spark DataFrame – 代码部分

➢ Spark DataFrame Documentation Example

```
# Schema - 自己设置

from pyspark.sql.types import (StructField,StringType,
                               IntegerType,StructType)

data_schema = [StructField('age',IntegerType(),True),
               StructField('name',StringType(),True)]
final_struc = StructType(fields=data_schema)

df = spark.read.json('file:///…/people.json',schema=final_struc)
df.printSchema()
```

*

字段名
字段类型
字段是否可为空

```
root
 |-- age: integer (nullable = true)
 |-- name: string (nullable = true)
```

Spark DataFrame

❑ Spark DataFrame – 代码部分

➢ Spark DataFrame Documentation Example

```
# Schema - 自己设置

# 写在一起
Schema = StructType([
                     StructField('age',IntegerType(),True),
                     StructField('name',StringType(),True),
])
# print(Schema))

df = spark.read.json('file:///… …/people.json',schema=Schema)
df.show()
```

```
+----+-------+
| age|   name|
+----+-------+
|null|Michael|
|  30|   Andy|
|  19| Justin|
+----+-------+
```

Spark DataFrame

❑ Spark DataFrame – 代码部分

➢ Spark DataFrame Documentation Example

```
# 一些基本操作 - 1（获取、查看、查询数据）
```

```
df['age']
```

```
Column<b'age'>
```

```
type(df['age'])
```

```
pyspark.sql.column.Column
```

```
df.select('age')
```

```
DataFrame[age: int]
```

```
df.select('age').show()
```

```
+----+
| age|
+----+
|null|
|  30|
|  19|
+----+
```

```
df.select(['age','name']).show()
```

```
+----+-------+
| age|   name|
+----+-------+
|null|Michael|
|  30|   Andy|
|  19| Justin|
+----+-------+
```

Spark DataFrame

❑ Spark DataFrame – 代码部分

➢ Spark DataFrame Documentation Example

```
# 一些基本操作 - 1（获取、查看、查询数据）
df.head(2)

df.head(2)[0]
```

```
df.head(2) # return [row0 row1]
```

```
[Row(age=None, name='Michael'), Row(age=30, name='Andy')]
```

```
df.head(2)[0]
```

```
Row(age=None, name='Michael')
```

```
# 创建新列
df.withColumn('double',df['age']*2).show()
```

```
+----+-------+------+
| age|   name|double|
+----+-------+------+
|null|Michael|  null|
|  30|   Andy|    60|
|  19| Justin|    38|
+----+-------+------+
```

```
# 重命名
df.withColumnRenamed('age','my_new_age').show()
```

```
+----------+-------+
|my_new_age|   name|
+----------+-------+
|      null|Michael|
|        30|   Andy|
|        19| Justin|
+----------+-------+
```

Spark DataFrame

❑ Spark DataFrame – 代码部分

➢ Spark DataFrame Documentation Example

```
# 一些基本操作 - 1（获取、查看、查询数据）

# 对DataFrame使用SQL查询，需要将其注册到临时视图
df.createOrReplaceTempView('people')

results = spark.sql("SELECT * FROM people ")
results.show()
```

```
+----+-------+
| age|   name|
+----+-------+
|null|Michael|
|  30|   Andy|
|  19| Justin|
+----+-------+
```

```
new_results = spark.sql("SELECT * FROM people WHERE age=30")
new_results.show()
```

```
+---+----+
|age|name|
+---+----+
| 30|Andy|
+---+----+
```

Spark DataFrame

❑ Spark DataFrame – 代码部分

➢ Spark DataFrame Documentation Example

```
# 一些基本操作 - 2 (filter)

from pyspark.sql import SparkSession
spark = SparkSession.builder.appName('ops').getOrCreate()

df=spark.read.csv("file:///.../appl_stock.csv",inferSchema=True,header=True)
```

```
# inferSchema: 读取json文件的时候没有这个选项，读取csv的时候有，它指的是spark自己判断数据类型
```

```
df.printSchema()
```

```
root
 |-- Date: timestamp (nullable = true)
 |-- Open: double (nullable = true)
 |-- High: double (nullable = true)
 |-- Low: double (nullable = true)
 |-- Close: double (nullable = true)
 |-- Volume: integer (nullable = true)
 |-- Adj Close: double (nullable = true)
```

Spark DataFrame

❑ Spark DataFrame – 代码部分

➢ Spark DataFrame Documentation Example

```
# 一些基本操作 - 2 (filter)

df.show() # Volume:交易量
```

```
+--------------------+------------------+------------------+------------------+------------------+---------+------------------+
|                Date|              Open|              High|               Low|             Close|   Volume|         Adj Close|
+--------------------+------------------+------------------+------------------+------------------+---------+------------------+
|2010-01-04 00:00:...|        213.429998|        214.499996|212.38000099999996|        214.009998|123432400|         27.727039|
|2010-01-05 00:00:...|        214.599998|        215.589994|        213.249994|        214.379993|150476200|27.774976000000002|
|2010-01-06 00:00:...|        214.379993|            215.23|        210.750004|        210.969995|138040000|27.333178000000004|
```

```
df.head(3)[0]


# 获取收盘价低于$500的所有数据

df.filter("Close<500").show()
```

```
+--------------------+------------------+------------------+------------------+------------------+---------+------------------+
|                Date|              Open|              High|               Low|             Close|   Volume|         Adj Close|
+--------------------+------------------+------------------+------------------+------------------+---------+------------------+
|2010-01-04 00:00:...|        213.429998|        214.499996|212.38000099999996|        214.009998|123432400|         27.727039|
|2010-01-05 00:00:...|        214.599998|        215.589994|        213.249994|        214.379993|150476200|27.774976000000002|
|2010-01-06 00:00:...|        214.379993|            215.23|        210.750004|        210.969995|138040000|27.333178000000004|
```

Spark DataFrame

❑ Spark DataFrame – 代码部分

➢ Spark DataFrame Documentation Example

```
# 一些基本操作 - 2 (filter)
df.filter("Close<500").select("Open").show()
```

```
+------------------+
|              Open|
+------------------+
|        213.429998|
|        214.599998|
|        214.379993|
|            211.75|
```

```
df.filter("Close<500").select(["Open","Close"]).show()
```

```
+------------------+------------------+
|              Open|             Close|
+------------------+------------------+
|        213.429998|        214.009998|
|        214.599998|        214.379993|
```

Spark DataFrame

❑ Spark DataFrame – 代码部分

➢ Spark DataFrame Documentation Example

```
# 一些基本操作 - 2 (filter)

df.filter(df['Close'] < 500).select('Volume').show()



# 多个条件
df.filter(df['Close'] < 200 and df['Open'] > 200).show()
```

报错:

```
ValueError: Cannot convert column into bool: please use '&' for 'and', '|' for 'or',
'~' for 'not' when building DataFrame boolean expressions.
```

Spark DataFrame

❑ Spark DataFrame – 代码部分

➢ Spark DataFrame Documentation Example

```
# 一些基本操作 - 2 (filter)

df.filter(df['Close'] < 200 & df['Open'] > 200).show()
```

报错:

```
Py4JError: An error occurred while calling o369.and. Trace:
py4j.Py4JException: Method and([class java.lang.Integer]) does not exist
```

```
# 解决办法: 加括号
df.filter( (df['Close'] < 200) & (df['Open'] > 200) ).show()


df.filter( (df['Close'] < 200) & ~(df['Open'] > 200) ).show()
```

Spark DataFrame

❑ Spark DataFrame – 代码部分

➢ Spark DataFrame Documentation Example

```
# 一些基本操作 - 2 (filter)

df.filter(df['Low'] == 197.16).show()
```

```
+--------------------+-----------------+----------+------+------+---------+---------+
|                Date|             Open|      High|   Low| Close|   Volume|Adj Close|
+--------------------+-----------------+----------+------+------+---------+---------+
|2010-01-22 00:00:...|206.78000600000001|207.499996|197.16|197.75|220441900|25.620401|
+--------------------+-----------------+----------+------+------+---------+---------+
```

```
df.filter(df['Low'] == 197.16).collect()
```

```
[Row(Date='2010-01-22', Open=206.78000600000001, High=207.499996, Low=197.16, Close=197.75,
Volume=220441900, Adj Close=25.620401)]
```

```
result = df.filter(df['Low'] == 197.16).collect()


row = result[0]
```

Spark DataFrame

❑ Spark DataFrame – 代码部分

➢ Spark DataFrame Documentation Example

```
# 一些基本操作 - 2
```

```
row.asDict()
```

```
{'Adj Close': 25.620401,
 'Close': 197.75,
 'Date': datetime.datetime(2010, 1, 22, 0, 0),
 'High': 207.499996,
 'Low': 197.16,
 'Open': 206.78000600000001,
 'Volume': 220441900}
```

```
row.asDict()['Volume']
```

Spark DataFrame

❑ Spark DataFrame – 代码部分

➢ Spark DataFrame Documentation Example

```
# 一些基本操作 - 3 (groupBy & agg  & orderBy )

from pyspark.sql import SparkSession
spark = SparkSession.builder.appName('aggs').getOrCreate()
df =
spark.read.csv("file:///.../sales_info.csv",inferSchema=True,header=True)
df.show()
```

```
+-------+-------+-----+
|Company| Person|Sales|
+-------+-------+-----+
|   GOOG|    Sam|200.0|
|   GOOG|Charlie|120.0|
|   GOOG|  Frank|340.0|
|   MSFT|   Tina|600.0|
```

```
df.printSchema()
```

```
root
 |-- Company: string (nullable = true)
 |-- Person: string (nullable = true)
 |-- Sales: double (nullable = true)
```

Spark DataFrame

❑ Spark DataFrame – 代码部分

➢ Spark DataFrame Documentation Example

```
# 一些基本操作 - 3 (groupBy & agg  & orderBy )
df.groupBy("Company") # groupyBy object: shows location and memory


df.groupBy("Company").sum().show()
```

mean, max, min, count

```
# Count
df.groupBy("Company").count().show()
```

```
+-------+-----+
|Company|count|
+-------+-----+
|   APPL|    4|
|   GOOG|    3|
|     FB|    2|
|   MSFT|    3|
+-------+-----+
```

```
# Mean
df.groupBy("Company").mean().show()
```

```
+-------+-----------------+
|Company|       avg(Sales)|
+-------+-----------------+
|   APPL|            370.0|
|   GOOG|            220.0|
|     FB|            610.0|
|   MSFT|322.3333333333333|
+-------+-----------------+
```

```
# Sum
df.groupBy("Company").sum().show()
```

```
+-------+----------+
|Company|sum(Sales)|
+-------+----------+
|   APPL|    1480.0|
|   GOOG|     660.0|
|     FB|    1220.0|
|   MSFT|     967.0|
+-------+----------+
```

Spark DataFrame

❑ Spark DataFrame – 代码部分

➢ Spark DataFrame Documentation Example

```
# 一些基本操作 - 3 (groupBy & agg  & orderBy )
df.agg({'Sales':'sum'}).show()
```

```
# 我们传入要“聚合”的列名，然后传入函数（字符串）
mean, max, min, count
```

```
# 和groupby一起使用
group_data = df.groupBy("Company")
group_data.agg({'Sales':'max'}).show()
#  same as : .groupBy.max
```

```
+-------+----------+
|Company|max(Sales)|
+-------+----------+
|   APPL|     750.0|
|   GOOG|     340.0|
|     FB|     870.0|
|   MSFT|     600.0|
+-------+----------+
```

Spark DataFrame

❑ Spark DataFrame – 代码部分

➢ Spark DataFrame Documentation Example

```
# 一些基本操作 - 3
# 函数
from pyspark.sql.functions import countDistinct,avg,stddev

df.select(countDistinct('Sales')).show()        # 销售值的不同数量
```

```
+--------------------+
|count(DISTINCT Sales)|
+--------------------+
|                  11|
+--------------------+
```

```
df.select(avg('Sales')).show()  # 平均销售额
```

```
+-----------------+
|       avg(Sales)|
+-----------------+
|360.5833333333333|
+-----------------+
```

Spark DataFrame

❑ Spark DataFrame – 代码部分

➢ Spark DataFrame Documentation Example

```
# 一些基本操作 - 3
df.select(avg('Sales').alias('Average Sales')).show()  # 也可以自己起名


df.select(stddev('Sales')).show()
```

```
+------------------+
|stddev_samp(Sales)|
+------------------+
|250.08742410799007|
+------------------+
```

```
from pyspark.sql.functions import format_number
sales_std = df.select(stddev("Sales").alias('std'))
sales_std.select(format_number('std',2).alias('std')).show()
```

Spark DataFrame

❑ Spark DataFrame – 代码部分

➢ Spark DataFrame Documentation Example

```
# 一些基本操作 - 3

# orderby
df.orderBy("Sales").show()
```

```
+-------+-------+-----+
|Company| Person|Sales|
+-------+-------+-----+
|   GOOG|Charlie|120.0|
|   MSFT|    Amy|124.0|
|   APPL|  Linda|130.0|
```

```
df.orderBy(df["Sales"].desc()).show()
```

```
+-------+-------+-----+
|Company| Person|Sales|
+-------+-------+-----+
|     FB|   Carl|870.0|
|   APPL|   Mike|750.0|
|   MSFT|   Tina|600.0|
```

Spark DataFrame

❑ Spark DataFrame – 代码部分

➢ Spark DataFrame Documentation Example

```
# 一些基本操作 - 4 （缺失数据）

from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("miss").getOrCreate()
df=spark.read.csv("...ContainsNull.csv",inferSchema=True,header=True)
```

```
+----+-----+-----+
|  Id| Name|Sales|
+----+-----+-----+
|emp1| John| null|
|emp2| null| null|
|emp3| null|345.0|
|emp4|Cindy|456.0|
+----+-----+-----+
```

```
df.show()
df.na.drop().show()
```

```
+----+-----+-----+
|  Id| Name|Sales|
+----+-----+-----+
|emp4|Cindy|456.0|
+----+-----+-----+
```

Spark DataFrame

❑ Spark DataFrame – 代码部分

➢ Spark DataFrame Documentation Example

```
# 一些基本操作 - 4 （缺失数据）
# 可以指定阈值
df.na.drop(thresh=2).show() # 该行至少需要有2个空值才被丢弃
```

```
+----+-----+-----+
|  Id| Name|Sales|
+----+-----+-----+
|emp1| John| null|
|emp3| null|345.0|
|emp4|Cindy|456.0|
+----+-----+-----+
```

```
df.na.drop(how='any').show() # 有任何空置，删除该行
```

```
+----+-----+-----+
|  Id| Name|Sales|
+----+-----+-----+
|emp4|Cindy|456.0|
+----+-----+-----+
```

```
df.na.drop(how='all').show() # 全部为空，删除该行
```

```
+----+-----+-----+
|  Id| Name|Sales|
+----+-----+-----+
|emp1| John| null|
|emp2| null| null|
|emp3| null|345.0|
|emp4|Cindy|456.0|
+----+-----+-----+
```

Spark DataFrame

❑ Spark DataFrame – 代码部分

➢ Spark DataFrame Documentation Example

```
# 一些基本操作 - 4 （缺失数据）

# subset 只考虑某个列
df.na.drop(subset=['Sales']).show()
```

```
df.na.fill('FILL VALUE').show()
```

```
+----+---------+-----+
|  Id|     Name|Sales|
+----+---------+-----+
|emp1|     John| null|
|emp2|NEW VALUE| null|
|emp3|NEW VALUE|345.0|
|emp4|    Cindy|456.0|
+----+---------+-----+
```

```
df.printSchema()
```

```
df.na.fill(0).show()
```

```
+----+-----+-----+
|  Id| Name|Sales|
+----+-----+-----+
|emp1| John|  0.0|
|emp2| null|  0.0|
|emp3| null|345.0|
|emp4|Cindy|456.0|
+----+-----+-----+
```

Spark DataFrame

❑ Spark DataFrame – 代码部分

➢ Spark DataFrame Documentation Example

```
# 一些基本操作 - 4 （缺失数据）
df.na.fill('No Name',subset=['Name']).show()
```

```
+----+-------+-----+
|  Id|   Name|Sales|
+----+-------+-----+
|emp1|   John| null|
|emp2|No Name| null|
|emp3|No Name|345.0|
|emp4|  Cindy|456.0|
+----+-------+-----+
```

```
# 也可以使用平均值进行填充
from pyspark.sql.functions import mean
mean_val = df.select(mean(df['Sales'])).collect()

mean_val[0]

mean_sales = mean_val[0][0]
```

```
400.5
```

Spark DataFrame

❑ Spark DataFrame – 代码部分

➢ Spark DataFrame Documentation Example

```
# 一些基本操作 - 4 （缺失数据）

df.na.fill(mean_sales,['Sales']).show()

# 可以写一行
df.na.fill(df.select(mean(df['Sales'])).collect()[0][0],['Sales']).show()
```

```
+----+-----+-----+
|  Id| Name|Sales|
+----+-----+-----+
|emp1| John|400.5|
|emp2| null|400.5|
|emp3| null|345.0|
|emp4|Cindy|456.0|
+----+-----+-----+
```

Spark DataFrame

❑ Spark DataFrame – 代码部分

➢ Spark DataFrame Documentation Example

```
# 一些基本操作 - 5 （日期）
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName('dates').getOrCreate()
df = spark.read.csv(“file///…/appl_stock.csv",header=True,inferSchema=True)

df.select(['Date','Open']).show()
```

Spark DataFrame

❑ Spark DataFrame – 代码部分

➢ Spark DataFrame Documentation Example

```
# 一些基本操作 - 5 （日期）
from pyspark.sql.functions import (dayofmonth,hour,
                                   dayofyear,month,
                                   year,weekofyear,
                                   format_number,date_format)

df.select(dayofmonth(df['Date'])).show()
```

```
+---------------+
|dayofmonth(Date)|
+---------------+
|              4|
|              5|
|              6|
+---------------+
```

```
df.select(hour(df['Date'])).show()
```

```
+----------+
|hour(Date)|
+----------+
|         0|
|         0|
```

```
df.select(month(df['Date'])).show()
```

```
+-----------+
|month(Date)|
+-----------+
|          1|
```

Spark DataFrame

❑ Spark DataFrame – 代码部分

➢ Spark DataFrame Documentation Example

```
# 一些基本操作 - 5 （日期）
# 每年的平均收盘价
# df.select(year(df['Date'])).show()
```

```
+----------+
|year(Date)|
+----------+
|      2010|
|      2010|
```

```
# 创建一个名为year的新列，它将是在DF日期调用的年份函数
df.withColumn('Year',year(df['Date'])).show()
```

```
+-------------------+-----------------+-----------------+-----------------+-----------------+---------+------------------+----+
|               Date|             Open|             High|              Low|            Close|   Volume|         Adj Close|Year|
+-------------------+-----------------+-----------------+-----------------+-----------------+---------+------------------+----+
|2010-01-04 00:00:...|        213.429998|        214.499996|212.38000099999996|        214.009998|123432400|         27.727039|2010|
|2010-01-05 00:00:...|        214.599998|        215.589994|        213.249994|        214.379993|150476200|27.774976000000002|2010|
```

```
newdf = df.withColumn('Year',year(df['Date']))
```

Spark DataFrame

❑ Spark DataFrame – 代码部分

➢ Spark DataFrame Documentation Example

```
# 一些基本操作 - 5 （日期）
#newdf.groupBy("Year").mean().show()
newdf.groupBy("Year").mean().select(['Year','avg(Close)']).show()
```

```
+---------+------------------+
|avg(Year)|        avg(Close)|
+---------+------------------+
|   2015.0|120.03999980555547|
|   2013.0| 472.6348802857143|
|   2014.0| 295.4023416507935|
```

```
result = newdf.groupBy("Year").mean().select(['Year','avg(Close)'])
result.show()
```

Spark DataFrame

❑ Spark DataFrame – 代码部分

➢ Spark DataFrame Documentation Example

```
# 一些基本操作 - 5 （日期）

result.withColumnRenamed("avg(Close)","Average Closing Price").show()


new = result.withColumnRenamed("avg(Close)","Average Closing Price")
new.select(['Year',format_number('Average Closing Price',2)]).show()


new.select(['Year',format_number('Average Closing Price',2).alias('Avg
Close')]).show()
```

```
spark.stop()
```